

Nest 进阶：企业级 Node.js 后端最主流框架 已付费

原创 神说要有光 神光的幸福生活 2026年6月11日 12:51 山东

业务项目的 Agent 都是跑在后端的，所以除了学习 Agent 框架外，我们还要学后端框架。

Node.js 最主流的后端框架是 Nest 。

有的同学可能会说，那 Express 呢？

Express 其实算不上一个真正的后端框架，它只是一个轻量的 HTTP 库、一个路由工具。

Express 没有模块化、没有统一架构、没有依赖注入、没有强类型约束、没有拦截器、没有统一的异常处理。

它几乎什么都不提供，只给你最基础的 req 和 res。

小 Demo、小接口用 Express 很快，但一上企业级 Agent 项目，立刻暴露致命问题：

代码乱、结构散、多个模块混在一起，后期根本维护不住。

而 Nest 是真正为企业级服务设计的完备框架，它自带一整套成熟架构：

模块化、依赖注入、TypeScript 强类型、统一生命周期、拦截器、管道、守卫、全局异常处理 。

这些能力，正是开发复杂 Agent 服务最需要、最刚需的。

Nest 能让整个服务结构清晰、可扩展、可维护、可上线。

Nest 并不是从零写的新框架。它底层默认封装的就是 Express，底层收发请求是靠 Express 支撑。

相当于 Nest 站在了 Express 的肩膀上，保留了它的生态，又补上了它缺失的架构、规范和工程化能力。

接下来我们就来学一下 Nest 特有的架构能力：

```
nest new nest-feature
```

在 Express 里，你需要手动 new 所有的对象

在 Nest 里不需要，它实现了 DI（依赖注入）

- 统一管理所有类（Controller、Service、Module、工具类）的实例
- 自动创建对象、自动注入依赖，不用开发者手动 new

我们在 @Module 的 providers 里配置的 provider：

用 @Injectable 声明的 class：

都可以自动根据名字或者 class 注入：

这就是 IoC（控制反转）或者叫 DI（依赖注入）

这个特性我们用过好多次了。

主要是来学一下 AOP（面向切面编程）

我们将系统比作一个纵向的、由多个业务模块（如用户模块、订单模块、支付模块）组成的结构。

而日志、权限、缓存、异常处理等功能，它们像一把“横向的刀”，切穿了所有的业务模块。

AOP 的本质就是：在代码运行的“特定位置”（切点），动态地插入这些横向逻辑。

如果没有 AOP，你可能需要在每一个 Controller 方法的开头都写一遍 `checkUserAuth()`。

这会导致：

- 维护噩梦：一旦权限校验逻辑变动，你需要修改项目里所有涉及该逻辑的文件。
- 代码污染：业务逻辑被大量的辅助代码包围，核心业务意图变得模糊。

Nest 的 AOP 可以在不修改原始代码的情况下，像“插件”一样为接口添加功能。

比如这样：

```
@Controller('orders')
@UseGuards(RolesGuard) // 这一行即实现了 AOP，为该类所有接口挂载了权限切面
export class OrdersController {
  @Get()
  @UseInterceptors(LoggingInterceptor) // 为单个接口挂载日志切面
  findAll() {
    // 这里只关心业务逻辑，不需要知道谁在校验权限，谁在记录日志
    return this.ordersService.findAll();
  }
}
```

通过 @UseGuards 装饰器给这个类所有接口加上了权限校验逻辑

通过 @UseInterceptors 装饰器给这个接口加了日志打印

这就是 AOP 的好处：

- 职责清晰：业务代码只做“业务”该做的事。
- 声明式编程：通过装饰器来描述需求（如 @Role('admin')），而不是通过命令式代码去实现需求。
- 集中化治理：所有的兜底异常处理（Exception Filters）和监控逻辑，可以在全局配置中一处修改，全站生效。

Nest 有这 4 种 AOP 的机制：

- 守卫（Guard）：请求进入控制器前执行，负责身份认证、权限校验，决定是否放行请求
- 管道（Pipe）：负责请求入参的校验、类型转换、数据清洗
- 拦截器（Interceptor）：环绕控制器方法执行，前置 / 后置处理，做日志、耗时统计、响应封装
- 异常过滤器（Exception Filter）：统一捕获程序异常，统一格式化返回错误信息

我们来用一下：

生成一个带 CURD 接口的 user 模块：

```
nest g res user
```

代码可以从仓库复制，这里我们用一遍各种 AOP 组件：

神光的编程秘籍

我们定义了两个 Pipe：

- ParsePositiveIntPipe：参数字符串转为正整数；非法值抛 400 BadRequestException

- ParseAgePipe: 将 age 查询参数字符串转为数字; 非法值抛 400

定义了一个 Guard:

- AuthGuard: 校验 Bearer Token, 并将当前用户信息挂到 request.user, token 有效且有权限就放行, 否则返回阻止访问返回无权限 403

定义了一个 Interceptor:

- TransformInterceptor: 打印请求、响应日志, 转换响应格式

定义了一个 ExceptionFilter:

- AllExceptionsFilter: 捕获所有异常, 统一错误格式的响应

还有一个自定义装饰器:

- @CurrentUser(): 从 request.user 读取当前登录用户

通过这个综合小实战, 你应该能体会到 AOP 的好处了。

把这些通用逻辑抽离出来, 用到的时候启用, 不用每个 controller 的 handler 里都写一遍。

这里我们还用到了几个取请求参数的装饰器:

- @Body(): 获取 POST、PUT 等请求的请求体数据
- @Param(): 获取路由路径参数
- @Query(): 获取 URL 后面的查询参数
- @Headers(): 获取请求头信息

这里我们还用到了 token, 一般现在常用的方案是 JWT

就是在 Authorization 的 header 里通过 Bearer xxx 携带:

token 可以解码出用户信息

校验通过后放到 request 对象上

前面是模拟实现的，接下来我们换成真实的 jwt：

```
pnpm install @nestjs/jwt
```

创建 jwt-test 模块：

jwt-test.module.ts

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { JwtTestController } from './jwt-test.controller';
import { JwtTestService } from './jwt-test.service';

@Module({
  imports: [
    JwtModule.register({
      secret: 'jwt-test-secret-key',
      signOptions: { expiresIn: '1h' },
    }),
  ],
  controllers: [JwtTestController],
  providers: [JwtTestService],
})
```



```
})  
exportclass JwtTestModule {}
```

jwt-test.service.ts

```
import { Injectable, UnauthorizedException } from '@nestjs/common';  
import { JwtService } from '@nestjs/jwt';  
  
export interface JwtTestPayload {  
  sub: number;  
  username: string;  
}  
  
@Injectable()  
exportclass JwtTestService {  
  constructor(private readonly jwtService: JwtService) {}  
  
  sign(payload: JwtTestPayload): string {  
    returnthis.jwtService.sign(payload);  
  }  
  
  verify(token: string): JwtTestPayload {  
    try {  
      returnthis.jwtService.verify<JwtTestPayload>(token);  
    } catch {  
      thrownew UnauthorizedException('Token 无效或已过期');  
    }  
  }  
}
```

jwt-test.controller.ts

```
import {  
  Body,  
  Controller,  
  Get,  
  Headers,  
  Post,  
  UnauthorizedException,  
} from '@nestjs/common';  
import { JwtTestService } from '../jwt-test.service';  
import type { JwtTestPayload } from '../jwt-test.service';  
  
@Controller('jwt-test')  
exportclass JwtTestController {  
  constructor(private readonly jwtTestService: JwtTestService) {}  
  
  @Post()  
  create(@Body() payload: JwtTestPayload): string {  
    returnthis.jwtTestService.sign(payload);  
  }  
  
  @Get()  
  verify(@Headers('token') token: string): JwtTestPayload {  
    returnthis.jwtTestService.verify(token);  
  }  
  
  @Get()  
  verifyToken(@Headers('token') token: string): boolean {  
    returnthis.jwtTestService.verifyToken(token);  
  }  
}
```

```

/** 签发 JWT */
@Post('sign')
sign(@Body() payload: JwtTestPayload) {
    const accessToken = this.jwtTestService.sign(payload);
    return { access_token: accessToken };
}

/** 校验 JWT 并返回 payload */
@Get('verify')
verify(@Headers('authorization') authorization?: string) {
    const token = this.extractBearerToken(authorization);
    if (!token) {
        throw new UnauthorizedException('请携带 Bearer Token');
    }

    return this.jwtTestService.verify(token);
}

private extractBearerToken(authorization?: string): string | null {
    if (!authorization) {
        return null;
    }

    const [type, token] = authorization.split(' ');
    if (type !== 'Bearer' || !token) {
        return null;
    }

    return token;
}
}

```

和之前的 JWT 流程一样，不过这次是用真正的 token

curl-test2.md

```

# 1. 签发 JWT → 200
curl -X POST http://localhost:3000/jwt-test/sign \
  -H "Content-Type: application/json" \
  -d '{"sub": 1, "username": "testuser"}'

# 2. 校验 JWT → 200 (把 <token> 换成第 1 步返回的 access_token)
curl http://localhost:3000/jwt-test/verify \
  -H "Authorization: Bearer <token>"

```

```
# 3. 未携带 Token → 401
curl http://localhost:3000/jwt-test/verify

# 4. Token 无效 → 401
curl http://localhost:3000/jwt-test/verify \
  -H "Authorization: Bearer invalid-token"
```

测一下：

神光的编程秘籍

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们过了一遍 Nest 的核心特性 IoC/DI、AOP。

依赖注入用过很多了，就是声明的 provider 可以在 @Inject 的地方自动注入，不用手动 new

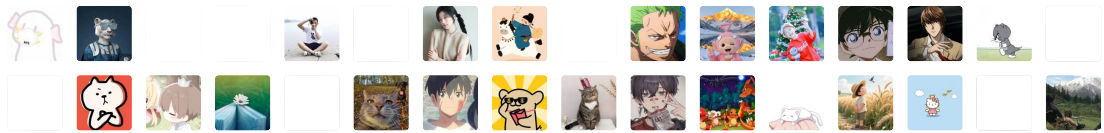
AOP 则是把通用逻辑抽离到 Pipe、Guard、Interceptor、ExceptionHandler 组件里，用到的时候动态加上。

我们还过了一下常用的 JWT 在 nest 里的实现。

如果你在 Express 里做这些还是挺麻烦的，没有模块化、没有这些组件的拆分。

而用 Nest 就可以很好的管理项目架构、规范代码的写法，这就是框架的意义。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇 · Mem0：分层记忆 + 三路召回的长期记忆方案

留言 14

写留言



LKY 四川 4小时前

光神, hono.js 和 nest.js 谁更适合agent 相关的后端部分的开发？前端开发来使用hono感觉就像在写前端一样，用nest就是有点不太习惯



神光的幸福生活 作者 4小时前
nest 是最主流的服务端框架



羊毛出在羊身上 广东 3天前
之前我买了光的Nest小册



希格斯 陕西 3天前
我也买了,没看完



神光的幸福生活 作者 3天前
回复 希格斯: AI 时代了，会基础就行，然后用到啥再说

1条回复



淘气包包 四川 3天前
ValidationPipe 现在这种内置的可以参数的校验，普通的就可以用它吧，特别的再写规则



神光的幸福生活 作者 3天前
是的，这里是讲基础



梦泉 江苏 3天前
直接用fastapi写吧，那个简单啊

 神光的幸福生活 作者 3天前
那个是 python, py后面讲

2条回复

 婧 四川 3天前
这不是你nest小册里的内容吗
首评

 神光的幸福生活 作者 3天前
因为 agent 课程也用 nest, 需要加一节讲一下基础